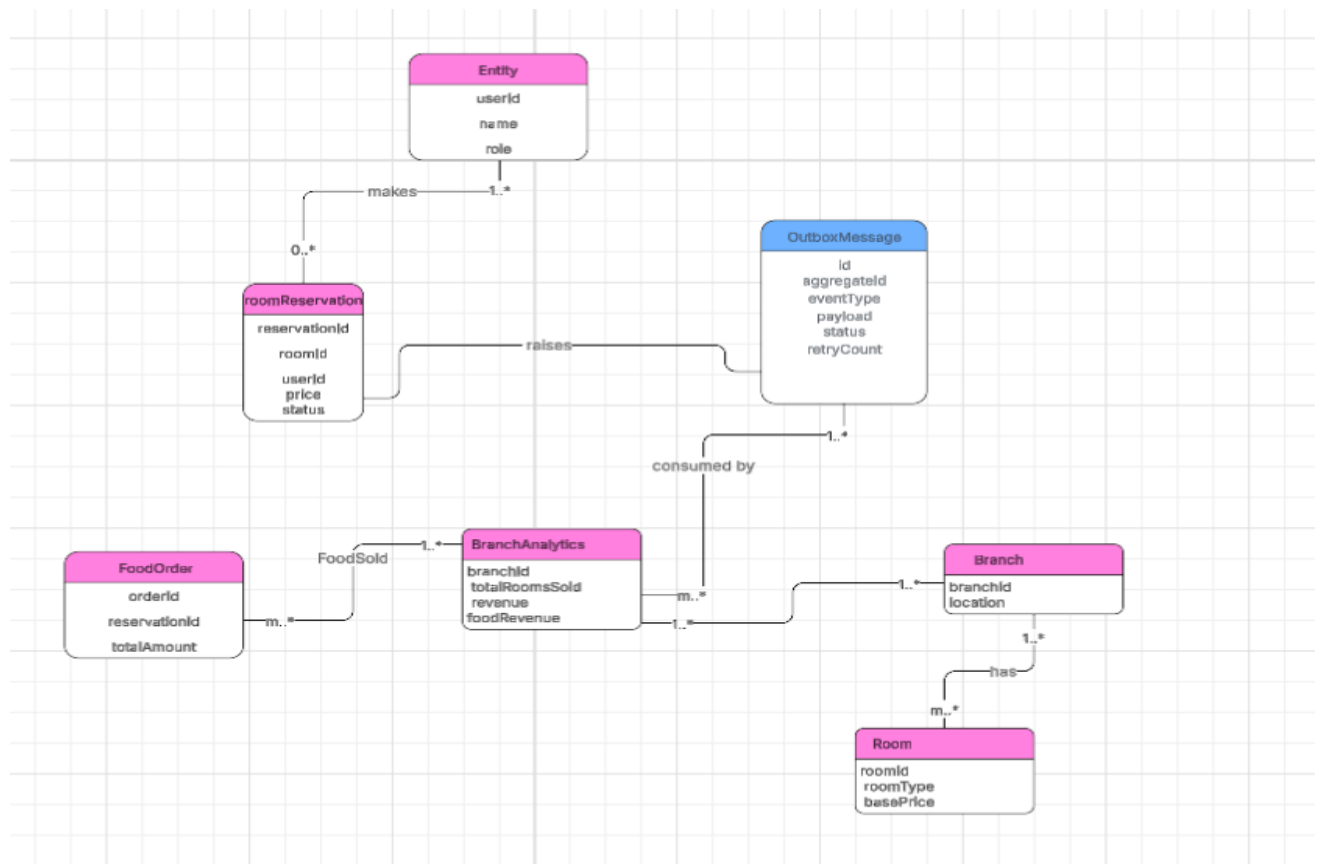




Outbox Documentation

Centralized Hotel Management System

1. Overview

The Centralized Hotel Management System implements a **Transactional Outbox Pattern** to guarantee reliable domain event delivery across modules. This ensures **eventual consistency**, even in cases of system failures or temporary unavailability of the message broker (RabbitMQ). The pattern supports the transition from a **modular monolith** to microservices while maintaining data integrity.

2. Purpose

The primary goals of implementing the Transactional Outbox are:

1. Ensure that **all domain events** generated by business actions are **persisted atomically** with aggregate changes.

2. Provide **reliable asynchronous communication** between modules (e.g., Reservation → Analytics → Customer Management).
3. Enable **retry mechanisms** for failed event deliveries without data loss.
4. Maintain **module isolation**, preventing direct database access between modules.

3. Mechanism

3.1 Domain Event Generation

- Aggregates emit **domain events** when important business operations occur.
Examples:
 - RoomReserved (Reservation & Pricing module)
 - PriceUpdated (Reservation & Pricing module)
 - FoodSold (Food & Beverage module)
- Domain events capture all information needed by other modules to react appropriately.

3.2 Outbox Table

- Each domain event is saved to the **Outbox table** within the **same database transaction** as the aggregate modification.
- Typical table structure:

Column	Description
Id	Unique identifier of the event
EventType	Type of the event (e.g., RoomReserved)
Payload	Serialized JSON of the event data
CreatedAt	Timestamp of event creation
ProcessedAt	Timestamp when the event was successfully published
RetryCount	Number of publishing attempts

- This approach ensures **transactional integrity**: if the database transaction fails, the event is not recorded, preventing inconsistency.

3.3 Event Publisher (Background Worker)

- A **Quartz.NET scheduled job** periodically polls the Outbox table for unprocessed events.
- For each event:
 1. Serialize and publish it to **RabbitMQ**.
 2. On success, mark the event with `ProcessedAt`.
 3. On failure, increment `RetryCount` and leave the event for the next scheduled attempt.

3.4 Retry Mechanism

- Failed events are retried automatically with **exponential backoff**.
- Events exceeding the maximum retry threshold are logged for **manual inspection**.
- This ensures **no events are lost** if the broker or network is temporarily down.

4. Module Communication

- **Event-Driven:** Modules communicate **exclusively through domain events**.
- Example workflow:
 - Reservation module publishes `RoomReserved`.
 - Analytics module consumes `RoomReserved` to update occupancy metrics.
 - Customer Management module consumes `RoomReserved` to update guest history.
- This ensures **decoupling**, **scalability**, and **future-proofing** for microservices.

5. Security Considerations

- Outbox events are stored internally and are **not exposed directly** to external clients.
- Communication with RabbitMQ is secured using **OAuth2/OpenID Connect tokens** from **Keycloak**.
- Role-based access control ensures only authorized modules can consume or publish events.

6. Benefits

1. **Reliability:** Guarantees domain events are delivered without loss.
2. **Consistency:** Maintains eventual consistency across bounded contexts.
3. **Decoupling:** Modules remain isolated; no direct database sharing.
4. **Scalability:** Supports future migration to microservices without changing domain logic.
5. **Resilience:** Retry mechanisms handle temporary failures gracefully.

7. Summary

The Transactional Outbox in the Centralized Hotel Management System ensures that all domain events generated within the modular monolith are **persisted reliably, published asynchronously, and retried on failure**. This mechanism enables **event-driven communication** across modules, enforces **aggregate consistency**, and prepares the system for a smooth transition to microservices architecture.